



Informe Trabajo práctico Autómatas, Teoría de Lenguajes y Compiladores

Comisión S

Integrantes:

- Santiago Nogueira (64113)
- Santino Pepe (64147)
- Manuel Araujo Feltrin (64090)
- Maria Victoria Alvarez (63165)

Docentes a cargo:

- Arias Roig, Ana Maria
- Golmar, Mario Agustín

Fecha de Entrega: 28/11/2025

Tabla de Contenidos

1. Introducción.....	2
2. Modelo Computacional.....	2
2.1. Dominio.....	2
2.2. Lenguaje.....	2
3. Implementación.....	3
3.1. Frontend.....	3
3.2. Backend.....	4
3.3. Dificultades Encontradas.....	4
3.4. Ejemplo de implementación.....	5
4. Futuras Extensiones.....	7
5. Conclusiones.....	7
6. Referencias.....	8
7. Bibliografía.....	8

1. Introducción

En el presente informe se detallarán los contenidos principales para la correcta implementación de un compilador base para un **Lenguaje Específico de Dominio (DSL)**. El objetivo principal que se propuso fue diseñar un lenguaje declarativo que permita a los usuarios definir y validar flujos de procesamiento de datos (*data pipelines*) para procesos ETL/ELT.

En el proyecto se utilizaron Flex y Bison como las herramientas de construcción de compiladores y se utilizó el lenguaje C para el desarrollo. El proyecto se separó en tres partes. La primera donde se diseñó la idea y se elaboró la propuesta pensando en los requerimientos, casos de prueba y ejemplos. En la segunda etapa, se realizó el desarrollo del frontend donde se hizo el analizador léxico y sintáctico. Por último, en la tercera parte se terminó algunos aspectos de frontend pero principalmente se desarrolló el backend para la correcta resolución del compilador.

2. Modelo Computacional

2.1. Dominio

Se operó en el dominio de la Ingeniería de Datos y Business Intelligence. El modelo permite definir los cuatro componentes fundamentales de un *pipeline*:

- **param:** Parámetros externos para generar *pipelines* reutilizables (ej: rutas, fechas).
- **source:** Definición de fuentes de datos (inicialmente CSV).
- **dataset/transform:** Creación de conjuntos de datos y aplicación de transformaciones encadenadas.
- **sink:** Definición de destinos para la escritura de resultados (inicialmente CSV).

Las transformaciones se simulan sobre *datasets* en memoria, utilizando un backend simplificado.

2.2. Lenguaje

El lenguaje que se ha utilizado para la implementación del compilador es C.

- **Análisis Léxico (Flex):** Define tokens para palabras clave (param, source, filter, write, into, and, or), identificadores, literales de cadena, números enteros y operadores.

- **Análisis Sintáctico (Bison):** La gramática libre de contexto (BisonGrammar.y) define la jerarquía de las declaraciones (statements) y la precedencia de operadores aritméticos (+, -, *, /).
 - **Sintaxis Clave (DSL):**
 - Declaraciones: `param name = "value", source name { ... }.`
 - Transformación: `transform new_ds = from source_ds use filter { condition }.`
 - Flujo: `write dataset into sink.`
 - **Tipos de Expresiones:** Soporte para constantes (INTEGER, STRING_LITERAL, TRUE_TOKEN, FALSE_TOKEN, NULL_TOKEN), expresiones aritméticas y comparaciones básicas (<, >, =).
-

3. Implementación

3.1. Frontend

En el frontend se implementan las tres principales etapas: análisis léxico, sintáctico y semántico.

- **Léxico:** se utilizó Flex (FlexPatterns.l, FlexActions.c) para tokenizar la entrada. Controlando contextos especiales, como los comentarios multilínea (`/* ... */`).
- **Sintáctico:** se usó Bison (BisonGrammar.y, BisonActions.c). Su acción principal es construir el Árbol de Sintaxis Abstracta (AST), cuya estructura se define mediante *unions* y *structs* anidadas en AbstractSyntaxTree.h (ej. StatementList, TransformDeclaration, Expression).
- **Estado del Compilador:** El AST resultante (Program *) se almacena en el CompilerState global, haciéndolo accesible para las fases posteriores.

3.2. Backend

La lógica de ejecución de la compilación se encuentra en `EntryPoint.c`, cumpliendo con el deber de ocuparse de las fases de análisis y generación.

- **Análisis Semántico:** se ha implementado en `SemanticAnalyzer.c`, el cual se ejecuta *después* del análisis sintáctico.
 - **Primera Pasada:** se construye una Tabla de Símbolos (`SymbolTable`) para registrar todas las declaraciones (parámetros, fuentes, datasets, etc.) y evitar declaraciones duplicadas.
 - **Segunda Pasada:** se validan las dependencias y tipos de uso de los identificadores. Por ejemplo, se verifica que una *transform* solo se aplique a un *dataset* o *transform* declarado previamente, y que una instrucción *write* apunte a un *sink*.
- **Generación de Código:** se implementó en `Generator.c`, el cual se encarga de recorrer el AST y generar los archivos de salida:
 - **pipeline.dot:** se definen nodos con formas específicas (caja para *source/sink*, elipse para *dataset*, diamante para *transform*) y se dibujan las aristas de dependencia.
 - **manifest.json:** se exporta un JSON con listas de parameters, nodes (name, kind) y edges (from, to).

3.3. Dificultades Encontradas

Las dificultades iniciales encontradas a lo largo del desarrollo del proyecto se centran en la complejidad del análisis de dependencias:

- **Errores Sintácticos Tempranos:** Una dificultad que se encontró fue el poder garantizar que se rechace las construcciones inválidas en una etapa temprana por la parte sintáctica. Se necesitó ver casos al momento de tener punto y coma o identificadores aislados sin un contexto válido.
- **Errores en Análisis Semántico:** Al momento de implementar el análisis semántico, se tuvo problemas con la inserción y búsqueda inconsistente en la tabla de símbolos esto generaba errores de compilación cuando los había. También se tuvo problemas con las ambigüedades de tipos al no propagar correctamente el tipo de expresiones.

- **Compatibilidad del AST en el Desarrollo del Backend:** Otro desafío surgió al desarrollar el backend, debido a la necesidad de asegurar que las estructuras del AST definidas en el frontend sean consistentes con lo que espera el backend.

3.4. Ejemplo de implementación

Para la probar la correcta implementación del proyecto se deberá ejecutar el siguiente comando:

```
./run.sh <nombre_del_archivo>
```

Utilizando el ejemplo de código 3 del Stage I y luego de ejecutar el comando mencionado, el compilador devuelve el siguiente resultado:

```
digraph pipeline {  
  
    "sales_src" [shape=box, style=filled, fillcolor="lightblue"];  
    "sales" [shape=ellipse, style=filled, fillcolor="lightgreen"];  
    "sales_src" -> "sales";  
  
    "clean_sales" [shape=diamond, style=filled, fillcolor="yellow"];  
    "sales" -> "clean_sales";  
  
    "stores_src" [shape=box, style=filled, fillcolor="lightblue"];  
    "stores" [shape=ellipse, style=filled, fillcolor="lightgreen"];  
    "stores_src" -> "stores";  
  
    "sales_by_region" [shape=diamond, style=filled, fillcolor="yellow"];  
    "clean_sales" -> "sales_by_region";  
  
    "rank_in_region" [shape=diamond, style=filled, fillcolor="yellow"];  
    "sales_by_region" -> "rank_in_region";  
  
    "rank_report" [shape=box, style=filled, fillcolor="orange"];  
    "rank_in_region" -> "rank_report";  
  
}
```

```

{
  "parameters": [
    { "name": "data_dir" },
    { "name": "out_dir" }
  ],
  "nodes": [
    { "name": "sales_src", "kind": "source" },
    { "name": "sales", "kind": "dataset" },
    { "name": "clean_sales", "kind": "transform" },
    { "name": "stores_src", "kind": "source" },
    { "name": "stores", "kind": "dataset" },
    { "name": "sales_by_region", "kind": "transform" },
    { "name": "rank_in_region", "kind": "transform" },
    { "name": "rank_report", "kind": "sink" }
  ],
  "edges": [
    { "from": "sales_src", "to": "sales" },
    { "from": "sales", "to": "clean_sales" },
    { "from": "stores_src", "to": "stores" },
    { "from": "clean_sales", "to": "sales_by_region" },
    { "from": "sales_by_region", "to": "rank_in_region" },
    { "from": "rank_in_region", "to": "rank_report" }
  ]
}

```

Esto se traduce y produce el resultado de:

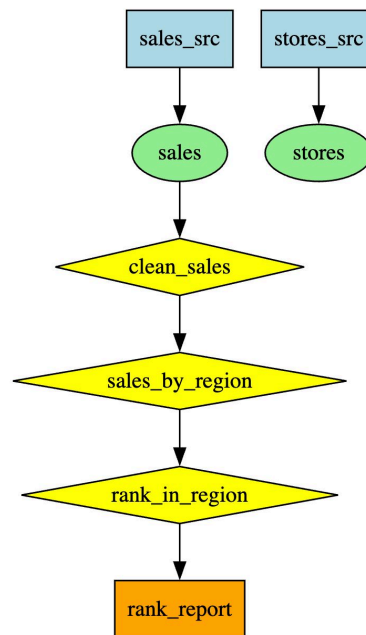


Figura 1: salida luego de ejecutar el código 3.

4. Futuras Extensiones

Para futuras extensiones se ha pensado en convertir el DSL en una herramienta completa para *data pipelines*.

- **Nuevas Fuentes y Destinos:** Ampliar el soporte más allá de CSV para incluir tablas SQL, formatos de columna como Parquet, APIs y JSON.
- **Múltiples salidas:** Permitir la escritura en múltiples *sinks* en un mismo *pipeline*.

5. Conclusiones

Se ha logrado el objetivo de construir un compilador para un Lenguaje Específico de Dominio, centrándose en el **Análisis Léxico, Sintáctico y Semántico básico**, fundamentales para garantizar la validez estructural y referencial del *pipeline*.

La generación de artefactos DOT y JSON cumple con los requisitos de la especificación y ha permitido ser una herramienta para la documentación y la visualización del flujo de datos.

Aunque la ejecución real de las transformaciones recae en un *backend* simplificado externo (como es Pandas), el compilador cumple su rol esencial de validar el diseño del *pipeline*, rechazando programas malformados antes de la ejecución.

6. Referencias

- **Flex y Bison:** Herramientas para la construcción de los analizadores léxico y sintáctico.
- **CMake:** Sistema de automatización de la construcción del proyecto.
- **Docker/Docker Compose:** Plataforma de contenedores utilizada para la gestión del entorno de desarrollo.
- **Graphviz (DOT):** Formato utilizado para la visualización del grafo de dependencias.

7. Bibliografía

- [TPE TLA: Stage I - Dataly](#)
- [Pandas](#): Librería de Python.